A smaller validated proof presentation of iPRA

Context as checking phase, and independence of equation progress

Research report prepared for Vladimir Reshetnikov

12 November 2026

Abstract

We audit the ninety proposals in *ConciseIPRA* at commit `62396b06020f`. The positive result is a smaller complete checker for the existing native Eq/D presentation: an empty retained context marks public-syntax validation, and adjoining a neutral assumption changes to proof checking. A separate phase flag disappears. The immutable-context implementation has 327 significant tokens, 375 AST nodes and 643 normalized UTF-8 bytes, compared with 333, 379 and 656 for the smallest matched native checker found in the audited sources. It accepts the same certificates and retains the same bi-interpretation with intuitionistic primitive recursive arithmetic. This is an implementation-level simplification of an established theory.

A separate written theorem proves that the native equation-progress rule is independent of the other seven kernel rules. The proof adapts a saturated-model construction and explicitly enforces coherence for raw iterator binders. Neither result proves absolute minimality. In particular, the new full native checker is larger than the corpus's compact protocols that take canonical guarded public syntax as an input contract. Thus the strongest request to beat the entire existing complexity envelope remains unmet; the report records the precise improvement and the remaining gap.

Contents

1 Result and comparison boundary **3**

2 Complete mathematical theory **3**
- 2.1 Numerical terms and formulas 4
- 2.2 All primitive axioms and inference rules 4
- 2.3 The ordinary realization and both interpretations 5

3 Removing the separate checking-phase flag **5**
- 3.1 The implementation invariant 5
- 3.2 Grammar and scope are still fully checked 6

4 Executable evidence and reproducibility **7**

5 The progress rule cannot simply be deleted **7**

6 What remains to achieve the requested target **10**

A Complete checker listing **11**

B Attribution and inherited proof appendices **12**

C Complementary introductions with one local assumption **12**
C.1 Complete native syntax and rule inventory . . . 12
C.2 Exact equivalence and the unused-witness construction . . . 13
C.3 Induction and numerical cases separate cleanly . . . 14
C.4 The entire ordinary logical interface . . . 15
C.5 Whole-domain realizations and both formula composites . . . 16
C.6 What the theorem does and does not claim . . . 17

D Arithmetic strength and both ordinary interpretation composites **17**
D.1 The arithmetic dependency order . . . 18

E Internal scalar predecessor bootstrap **19**
E.1 Elementary terms and two finite cycles . . . 19
E.2 A ramp with a global scaling identity . . . 20
E.3 A comparator from scalar iteration . . . 20
E.4 Saturated iteration gives predecessor . . . 21
E.5 Dependency boundary . . . 22

F From literal-equation induction to full intuitionistic PRA **22**
F.1 The exact hypotheses . . . 23
F.2 Constructor separation and two elementary tests . . . 23
F.3 Four elementary uses of equation induction . . . 24
F.4 An equation invariant that reflects actual equality . . . 25
F.5 Compilation of all quantifier-free formulas . . . 26
F.6 Recovery of induction and the full theory . . . 27

G Pairing, primitive recursion, and internal numerical composites **27**
G.1 Pairing from a bounded square-root search . . . 28
G.2 Compiling each primitive recursion . . . 28
G.3 Why both numerical round trips are internal . . . 29

1 Result and comparison boundary

The requested measure is the size of a straightforward complete proof checker. This makes the statement of the input contract part of the mathematical comparison. Counting a normalizer, parser or public-language validator in one program but outside another would not establish a reduction. All measurements below include each complete standalone Python module after `ast.unparse`, with one final LF. Comments and whitespace do not count as significant tokens. The runtime is CPython 3.12.14, matching the repository convention. No acceptance code is imported from a project helper; the checkers use only the standard JSON module.

Program or coordinate minimum	Tokens	AST nodes	Bytes
Frozen native public checker	333	379	656
New immutable-context checker	327	375	643
Same checker with explicit `type` test	327	375	646
Optional one-shot list implementation	325	375	641
Compact token minimum found	256	283	594
Compact AST minimum found	263	276	580
Compact byte minimum found	267	303	506

The new primary checker decreases the three matched coordinates by 6, 4 and 13 respectively. The compact minima are an empirical coordinatewise envelope, not one fictitious program of size $(256, 276, 506)$ and not a proved lower bound on all possible theories. Those compact programs do not themselves enforce the whole native public-goal grammar. The 643-byte program does. This distinction explains the useful matched improvement; it does not make 643 less than 506.

The comparison pins actual source files, not only README summary rows. The repository README lists ninety packages, while some smaller optional implementations appear only inside package subdirectories. The audit found the 506-byte implementation in `deterministic_introduction/implementation/code/compact_optional.py` and the 656-byte native control in the same directory's `native.py`. The archive freezes these files, the token and AST controls, their hashes, the source-census candidates and the new measurements. Short build scripts and programs importing their checking work from another file are not counted as standalone checkers.

A final comparison with repository head `09b3185117a5` found no changes under `ConciseIPRA/Research`; the measured corpus therefore remained unchanged. The comparison receipt is included in the archive.

Exact status. The mathematical theory below is the inherited scalar native theory. The new proof presentation reduces its implementation state. There is no newly deleted arithmetic axiom, no claim to improve the three-family logical inventory of other proposals, and no general minimality theorem. The independence theorem in Section 5 settles one specific deletion question, rather than a checker-size bound. All new arguments are written proofs with executable validation; they are not new Lean or Rocq verifications.

2 Complete mathematical theory

We specify the theory independently of the implementation. Its ordinary realization is bi-interpretable with full first-order iPRA, not merely with a quantifier-free fragment or the standard model.

Here iPRA means intuitionistic first-order arithmetic with one function symbol for each fixed primitive recursive description, its defining equations, successor injectivity and separation

from zero, and induction for quantifier-free formulas with parameters and side assumptions. Quantified formulas and all ordinary intuitionistic logical operations are part of the language; induction is restricted to the stated class.

2.1 Numerical terms and formulas

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the last child h of an iterator binds u. The counter and initial value are outside that binding. The update may contain passive numerical parameters. There is no quantification over function descriptions and no uniform evaluator for all primitive recursive programs.

Public formulas are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A.$$

The proof language also permits the auxiliary closure $\mathsf{Q}_y A$. The designated contradiction is $0 = S0$. In ordinary intuitionistic logic,

$$(s = t)^\circ = (s = t), \qquad (\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y),$$

$$(\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

A context is a finite reusable list of assumptions. Bound names are renamed before capture-avoiding substitution.

2.2 All primitive axioms and inference rules

There are two arithmetic computation axiom schemes, available in every context:

$$I(0, a; u.h) = a,$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The logical and equational inferences are the following. Together with the two computations these give eight primitive rule families for this presentation.

$$\frac{A \in \Gamma}{\Gamma \vdash A}\ \text{Hyp}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \text{Tr},$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \text{QE}, \qquad \frac{\Gamma \vdash \mathsf{D}_{x,y}B \qquad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A}\ \text{DE},$$

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \text{DI by cases}, \qquad \frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}E(x, y)}\ \text{DI by progress}.$$

Here F is any native formula template. In DE, b is fresh for the retained context, the conclusion and the fixed parameters of B. In both introductions z is fresh for the context, the witness t and the fixed parameters of the matrix. Progress is permitted only when E is literally an equation. Each displayed extra assumption is discharged.

There are no additional primitive structural rules, successor axioms, reflexivity axiom, ordinary propositional axioms, or unrestricted induction schema. Weakening, cut and substitution are admissible. Ordinary logical operations are derived macros. The repository's Lean kernel uses the equivalent one-term crossed transport rule and introduces $\mathsf{Q}_y E(y)$ directly by progress. Its theorem `iprf_iff_nprf` connects that kernel to the rule inventory above. A direct closure is interderivable with a paired formula whose existential coordinate is unused.

2.3 The ordinary realization and both interpretations

Let T_I be intuitionistic first-order logic with actual equality over the staged iterator descriptions, both computation schemes, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The explicit second formula connects designated numerical contradiction to genuine logical falsity. It prevents an unnoticed singleton model of the positive equations from being mistaken for the intended ordinary theory.

The native calculus and T_I have mutually faithful formula translations. Numerical cases and the equation-induction rule are derived from the two introduction modes. The selector $C(v, a, b) = I(v, a; u.b)$ is used to construct implication, conjunction, disjunction and the quantifiers. Their complete inherited derivations are reproduced in Appendix C.

For the arithmetic interpretation $T_I \to$ iPRA, interpret every fixed iterator by primitive recursion with an update that ignores the recursion counter. For iPRA $\to T_I$, first construct predecessor using only scalar iteration, cases and equation induction; then obtain equality reflection, quantifier-free induction, pairing and projections. For a primitive recursion $f(0, \vec{p}) = g(\vec{p})$ and $f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$, use

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The pairing functions are derived before this compiler is used. Internal equation induction proves the required counter and projection identities. External induction on each finite function description, followed by internal uniqueness of recursion, proves that translating twice preserves that function. Both numerical interpretation domains are the whole domain, equality is actual equality, and the graph of each domain composite is $x = y$. Both formula composites are provably equivalent to the original formula. These are the bi-interpretability assertions, beyond equiconsistency.

The inherited complete arithmetic proof is included in the appendices. The pinned Lean endpoints `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff` are supplied as reference source. They are not claimed as a new formalization of the Python implementation.

3 Removing the separate checking-phase flag

The control has a parameter v. Only the first call sets $v = 1$; this makes its opening traversal validate the full public goal grammar. Immediately afterward it sets $v = 0$, and all recursive calls use the ordinary proof-checking traversal. Contexts initially contain a neutral administrative assumption represented by the scalar 0.

The new checker starts with the empty tuple. While it is empty, the same traversal validates the public goal. Before consuming the first proof packet it adjoins the existing neutral assumption. Every subsequent checking context is nonempty. Thus the context already contains the information represented by v.

3.1 The implementation invariant

1. An empty context occurs only on entry to the public driver.
2. That driver accepts only a closed canonical public formula, which is a nonempty array and cannot close by the neutral hypothesis.
3. Before any proof-packet processing, the checker adds a neutral zero. Every recursive call consequently receives a nonempty context.

4. An additional neutral zero may be inserted on later non-hypothesis calls. The older context already contains zero, so this changes no membership test or mathematical assumption.

Contexts are immutable tuples. The scope floor k remains an independent integer. It is not recomputed from context length: duplicate neutral assumptions make context length unsuitable as a variable-scope measure. The decrement of k occurs at the original point before the fresh-variable opening. No fresh numerical parameter, witness or new logical axiom is introduced by this change.

3.2 Grammar and scope are still fully checked

The raw array codec stores $\mathsf{D}B$ as a unary array, an equation as a binary array, a successor as a unary term array, and an iterator as a ternary term array. The opening traversal adds $5 - \text{arity}$ to its depth. Starting at formula depth zero, the parity of the depth records the current sort: even for a formula, odd for a term. Consequently the permitted public array lengths are

$$\{1,\ 2 + (d \bmod 2)\}.$$

An integer leaf occurs only at a term position and is either zero or an odd coordinate at least three. Its scope must also satisfy $0 \le a < d$ on the public scan. The internal floor k subsequently admits fresh negative rigid parameters. Invalid public arrays, scalar formulas, booleans, floating-point coordinates, strings, nulls and objects reject. The checker does not merely rely on a promise that the goal is well formed.

The shortest primary implementation recognizes exact integer leaves by `int(a) is a`. For ordinary JSON values under the pinned CPython runtime, conversion preserves an exact integer object, whereas a Boolean or float converts to a distinct integer object. A conversion error is a rejection. Strings and objects already fail the preceding numerical comparison. The explicit `type(a) is int` variant costs three more bytes, with the same token and AST counts, and isolates this runtime detail.

Theorem 3.1 (Acceptance preservation)**.** *Ignoring finite memory and recursion limits, the new immutable-context checker accepts exactly the same finite JSON certificate values as the frozen native checker. On a successful run both consume the same suffix of the packet sequence and leave the same unused prefix. In particular, no certificate renaming or proof translation is required.*

Proof. Compare the initial scans. Empty-context tests in the candidate have the same truth values as the control's validation flag. The public grammar and scope conditions coincide, including rejection of scalar formula padding. A valid public formula is not the neutral scalar, so neither scan closes by hypothesis. The candidate then inserts zero and the control disables its flag.

For subsequent calls, relate contexts by their membership observations, allowing duplicate neutral zero entries and the administrative aliases 0 and `False`. Nonempty candidate contexts disable the extra syntax tests, just as $v = 0$ does in the control. The candidate's even-depth scalar fallback can return `False` where the control returns 0. All subsequent kernel observations of this padding are unchanged: equality, truth testing, integer comparison, multiplication, and membership agree; public exact-type testing has already ended. At meaningful numerical positions the same image is opened and the same scope test is applied. Induction on each successful opening establishes the relation between its outputs.

Neutral insertion preserves the context relation. Every packet read, major-premise call, computation or introduction dispatch, equality test, floor decrement, and continuation then has the corresponding result. Immutable context extensions preserve the relation in both branches. Induction on finite successful checking trees gives both implications and the same packet consumption. Invalid initial scans reject on both sides. Differences in exception messages and physical resource limits are not part of the abstract acceptance assertion. □

Corollary 3.2 (Preserved strength)**.** *With the inherited native-codec soundness and completeness theorem, the new checker is a complete proof presentation of the theory in Section 2. Its ordinary realization is equiconsistent and bi-interpretable with* iPRA*, with both numerical domain composites given by* $x = y$*.*

Proof. Acceptance preservation identifies the proved public formulas with the control's public formulas. The theory, its formula translations and its two internal round trips are unchanged. Apply the inherited native bi-interpretation described above. □

4 Executable evidence and reproducibility

The archive contains the complete standalone checker, a readable implementation, an explicit-type variant, frozen controls, representative proof certificates, and a standard-library reproduction script. The validation checks the actual native control rather than treating a different guarded-profile fixture ledger as the expected result.

The central risks are the initial public grammar, the change of phase, fresh-variable scope and contextual branching. The tests therefore compare initial scans on arbitrary JSON trees and generated grammatical trees; compare acceptance and packet consumption on the frozen corpus; and execute the real command-line programs with and without Python's optimization flag. Positive certificates exercise computation, transport, closure elimination, both introductions and a used induction assumption. Negative cases include unproved $0 = S0$, future-variable capture, context leakage, malformed instructions and malformed public goals. The suite passed 4,065 public-goal comparisons across five implementations, 80 repeated public entries, 21 complete fixture comparisons, 256 certificate mutations, an unused-prefix check, and 140 actual command-line processes. Of the 21 native fixtures, 13 accept and eight reject. The per-case outcomes appear in `evidence/validation.json`.

Finite tests check implementation behavior; Theorem 3.1 is the universal acceptance argument. The independently written model-theoretic proof below supplies the independence theorem. No finite collection of numerical examples is used to establish bi-interpretability.

```
python3 reproduce.py
python3 code/checker.py < validation/fixtures/computation_zero_local.json
python3 build.py
```

The first command verifies frozen source hashes, regenerates the measurements and runs the validation. The final command requires pdfLaTeX. The delivered PDF has already been compiled and rendered for inspection.

5 The progress rule cannot simply be deleted

Let $\mathcal{N}^-$ be the native scalar kernel with Hyp, the two iterator computation equations, equality transport, QE, DE and DI by cases, but without QI by progress. Its raw terms and formulas are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h), \qquad A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

The intended readings remain

$$Q_y B = B(0) \to \forall y B(y), \qquad D_{x,y} B = \exists x \big(B(x, 0) \to \forall y B(x, y)\big).$$

We use an ordinary classical structure to refute derivability. A classical structure is also a one-world intuitionistic structure, so this is a legitimate independence argument for the intuitionistic calculus.

Theorem 5.1 (Independence of equation progress)**.** *Put $E(y) \equiv I(y, 0; u.u) = 0$. The full native kernel proves $Q_y E(y)$, $E(v)$ for a free numerical variable v, and the closed public Eq/D formula*

$$D_{x,y}\big(I(y, 0; u.u) = 0\big), \qquad x \text{ unused.}$$

The kernel $\mathcal{N}^-$ proves none of them. This remains true even if the ordinary target theory of $\mathcal{N}^-$ is augmented by successor injectivity, coverage, $0 \neq S0$, absence of all finite successor cycles, and extensionality of fixed iterator bodies.

The construction below adapts the saturation and elementary-chain method used by Friedman for primitive recursion without induction [2, Theorem 8 and Lemmas 24–25]. The extra coherence condition is stated explicitly because the native syntax uses a bound body, rather than an independent name for every recursion program. Consequently this appendix claims an application to the native kernel, not a new general model-theoretic method.

Lemma 5.2 (An orbit in a saturated structure)**.** *If M is countably saturated and $H : M \to M$ is a total function definable with finitely many parameters, there is a sequence $(a_j)_{j\in\mathbb{Z}}$ in M satisfying $H(a_j) = a_{j+1}$.*

Proof. The partial type

$$\{\exists z\ H^m(z) = x : m \in \mathbb{N}\}$$

is finitely satisfiable: for a finite collection with maximum exponent m, choose $x = H^m(z_0)$ for any z_0. Saturation supplies a_0 in all finite iterate images. Given such an a_{-k}, the type in x consisting of $H(x) = a_{-k}$ and the same image conditions is finitely satisfiable, since a_{-k} lies in the image of H^{m+1} for every m. Choose a_{-k-1} realizing it. Define the positive-index elements by forward iteration. Only countable types in one variable and finite parameter sets have been used. □

The coherence requirement. Use a first-order description language. For a finite update term $h(u, \vec{p})$ in earlier symbols, its iterator has a symbol $J_h(n, a, \vec{p})$, with equations

$$J_h(0, a, \vec{p}) = a, \tag{1}$$

$$J_h(Sn, a, \vec{p}) = h(J_h(n, a, \vec{p}), \vec{p}). \tag{2}$$

For every two update templates whose iterator symbols have been introduced, require

$$\forall \vec{p} \forall \vec{q}\,(\forall u\ h(u, \vec{p}) = k(u, \vec{q})) \ \longrightarrow \ \forall n \forall a\ J_h(n, a, \vec{p}) = J_k(n, a, \vec{q}). \tag{3}$$

This is an external family of first-order sentences, not an object-language function quantifier. It is stronger than the substitution coherence needed below.

Lemma 5.3 (Coherent extension by iterators)**.** *Suppose M is a countably saturated structure with an injective successor, 0 outside the successor range, coverage, and no finite successor cycles. Suppose an existing family of fixed-body iterator symbols satisfies (1)–(3). The same domain can be expanded by iterator symbols for every update term over its present language, preserving all three requirements and all previously interpreted symbols.*

Proof. The successor graph is one component of type $\mathbb{N}$, containing 0, and components of type $\mathbb{Z}$. Choose an origin in each $\mathbb{Z}$ component.

For each actual unary function H defined by a present update term and a tuple of parameters, first ask whether H is represented by an already interpreted iterator template at some parameter tuple. If it is, retain that iterator's values for every counter and seed. Condition (3) makes this independent of the chosen old representation.

For each remaining H, on the component of 0 use the standard finite iterations $H^m(a)$ starting at the supplied seed a. On each $\mathbb{Z}$ component choose a bi-infinite H-orbit using Lemma 5.2, and assign its entries to the counter component in successor order. The orbit may be chosen separately for each seed; choosing the same orbit for all seeds is also allowed.

The choices are indexed by actual functions H, not by their syntactic descriptions. Thus two new parameterized templates representing the same H receive the same iterator values, and a new template equal to an old one inherits the old values. This proves coherence, including new–old pairs. The zero equation holds in the $\mathbb{N}$ component; the successor equation holds there by finite iteration and in every other component by the orbit equation. Every required update function is definable in the structure before the expansion, so no circular interpretation has been used. □

An initial failure of induction. Let M_0 have underlying set the disjoint union $\mathbb{N} \sqcup \mathbb{Z}$. Successor is ordinary successor on each component, and zero belongs to $\mathbb{N}$. Introduce the identity-update iterator $J(n, a)$ by

$$J(n,a) = \begin{cases} a, & n \text{ belongs to the } \mathbb{N} \text{ component,} \\ Sa, & n \text{ belongs to the } \mathbb{Z} \text{ component.} \end{cases}$$

Then $J(0, a) = a$ and $J(Sn, a) = J(n, a)$, exactly the computation equations for $h(u) = u$. There is only one iterator template, so coherence is automatic. For an element c of the $\mathbb{Z}$ component, $J(c, 0) = S0 \neq 0$.

Closing the language. Starting with M_0, alternate the following two operations:

1. Take a countably saturated elementary extension in the current first-order language.
2. Apply Lemma 5.3 to add all missing iterator templates over that language.

Use inclusions for the elementary embeddings and take the union after countably many stages. Each language is countable, since its descriptions are finite strings. Every raw scalar term is included at some finite stage, by induction on its finite syntax. Each fixed computation or coherence sentence holds at its stage of introduction, is preserved at all later elementary extensions, and therefore holds in the union. More formally, for a fixed stage language the later reducts form an elementary chain. Expanding a structure does not change the values of old symbols. The equation $J(c, 0) = S0$ and $S0 \neq 0$ are preserved throughout.

Saturation may create new counter components on which $J(n, 0) = 0$; the construction does not assert that this equation defines exactly the externally standard numbers in the final model. Only the preserved counterexample c is needed.

Why this interprets the raw binder syntax. Translate a raw iterator $I(n, a; u.h)$ using the symbol for the finite translated body h and its passive parameters. This translation respects capture-avoiding substitution semantically. At the iterator case, the two ways of translating a substitution may use different template symbols, but the structural induction hypothesis says their actual unary update functions are pointwise equal. Condition (3) equates the two iterator values. The variable, zero and successor cases are immediate. It follows that equality transport in arbitrary raw term or formula templates is sound, including templates with nested iterator binders.

Proof of Theorem 5.1. In the constructed structure read native equality as actual equality and read Q, D by their displayed ordinary formulas. The computation rules hold by construction. Hyp and transport are sound by ordinary equality and substitution. QE and DE are sound by their ordinary logical readings. DI by cases is sound because every numerical element is zero or a successor; its eigenvariable condition permits universal generalization in the successor branch. Thus all rules of $\mathcal{N}^-$ are sound.

The zero instance $E(0)$ is true. At the preserved element c, $I(c, 0; u.u) = S0 \neq 0$, so $\forall y E(y)$ and $Q_y E(y)$ are false. The open formula $E(v)$ is false under the assignment $v = c$. For an unused witness variable x, the ordinary reading of $D_{x,y}E(y)$ is $\exists x\, Q_y E(y)$. The numerical domain is whole and nonempty, with the designated element 0, so this is equivalent to $Q_y E(y)$ and is false as well. None of the three formulas can therefore be derived in $\mathcal{N}^-$.

In the full kernel, zero computation proves $E(0)$, and successor computation gives

$$I(Sz, 0; u.u) = I(z, 0; u.u).$$

Transport under the assumption $E(z)$ gives $E(Sz)$. QI by progress now proves $Q_y E(y)$, and QE with $E(0)$ proves $E(v)$. In this QI profile, $D_{x,y}E(y)$ is also obtained by DI by cases at witness 0: under the temporary assumption $E(0)$, QE of the proved closure supplies $E(Sz)$. Equivalently, this is the usual native equivalence between $Q_y E(y)$ and its unused-witness D form. In the original Choice-DI profile, whose progress rule concludes a D formula, the same progressive branch directly introduces $D_{x,y}E(y)$ at witness 0. Thus the separator is in the public Eq/D language and does not depend on which of these equivalent progress-rule profiles is used. □

Scope of the obstruction. The result concerns deleting the progress principle while retaining the other stated primitives and their ordinary readings. It is not a lower bound on the size of an arbitrary bi-interpretable presentation. In particular it does not obstruct fusing progress with another rule, changing the term language, or transporting induction through a different interpretation. The construction is a written model-theoretic proof; it has not been checked by Lean or Rocq.

6 What remains to achieve the requested target

The context-phase change is a strict improvement under the full native public-goal contract. It leaves a clear gap to the separate compact envelope. A stronger positive result must either reduce the actual logical or numerical mechanisms enough to outweigh its checking costs, or find a simpler complete implementation with a carefully stated public-language contract. A short dispatcher that outsources syntax, normalization or arithmetic is not a complete comparison.

Several tempting combinations fail to establish such a result. Restricting both introduction witnesses and specialization targets needs a new naming construction; the existing arguments for the separate restrictions can depend on each other's unrestricted operation. A counter-aware tied-seed recursor is already subject to a bounding-coordinate obstruction in the corpus. Finally, Theorem 5.1 rules out simply discarding the remaining equation-progress principle while keeping the other native rules. It permits fusing that principle into another rule or changing the interpretation, so it is not a claim that further simplification is impossible.

A Complete checker listing

The following is the entire primary acceptance module. The readable version in the archive expands the same control flow; it is not an unmeasured dependency.

```
import json

def c(a, g=(), k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] and (g or len
    (a) in (1, 2 + d % 2)) else a if k <= a < d and (g or (int(a) is a and d % 2 and (not a
    or a > 1 == a % 2))) else (d % 2 or not g) and i(t, d=3)
    if i(a) in g:
        return
    g += (0,)
    m, s, u, t = q()
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or
     (a := i(*a, t, 2))
    c(i(m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a, [(k := (k - 1))]),
     g + (i(h, (m or len(a) % 2 == u) and k),), k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

B Attribution and inherited proof appendices

The following five mathematical sections are reproduced from the pinned repository under its MIT No Attribution license, with their original attribution comments retained in the source files. They establish the native logical basis, ordinary bridge and scalar arithmetic construction. They are included to make the mathematical dependencies reviewable; they are not new results of this report. Their original local citation keys refer to the source series collected by the same pinned repository. The separate `reference/inherited_arity_checker.tex` records the earlier general codec proof and its sorted interpretation. The exact current-program dependency is the frozen native control plus its later source refinements; acceptance preservation above is proved directly against that frozen control.

C Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

C.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and

$\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

C.2 Exact equivalence and the unused-witness construction

Lemma C.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem C.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y) \rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y} E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition C.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

C.3 Induction and numerical cases separate cleanly

Lemma C.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma C.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0,t;u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

C.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v,a,b) := I(v,a;u.b)$ with u unused. Computation gives $C(0,a,b) = a$ and $C(Sv,a,b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s=t) := (C(v,0,s) = C(v,0,t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u,y \notin \mathrm{FV}(A)). \qquad \text{(Gate and padding)}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s=t, u=w) := (C(v,s,u) = C(v,t,w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A,B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE,B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A,G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A,B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A,B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A,B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0=1). \end{aligned} \qquad \text{(Logical macros)}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A,B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A,B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\,(x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

C.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem C.2 preserves every native sequent.

C.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

D Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section C recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

D.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem D.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

E Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [4] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

E.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{4}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{5}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

E.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{6}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{7}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{8}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) = G(3n) + N(3n) + 4 &= 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (8) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{9}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (8); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

E.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{10}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (9), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{11}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{12}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

E.4 Saturated iteration gives predecessor

We now need only the three equations (12). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (13) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (14) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{15}$$

The base is (14). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{16}$$

The first uses the comparator recurrence. The second uses (15) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{17}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{18}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (17) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

E.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

F From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [5]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix E, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

F.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{19}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{20}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{21}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{22}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{23}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (23) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

F.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{24}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{25}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{26}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (19). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (25); symmetry and (22) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{27}$$

$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{28}$$

$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{29}$$

To prove the range assertion, split (21). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (20) also give, for all u, v,

$$C(u, v, v) = v. \tag{30}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{31}$$

If $u = 1 = S0$, (26) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

F.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{32}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{33}$$

Using these equations and (23), we now derive

$$0 + y = y, \tag{34}$$

$$(Sx) + y = S(x + y), \tag{35}$$

$$d(Sx, Sy) = d(x, y), \tag{36}$$

$$d(Sa, a) = 1. \tag{37}$$

Each induction below has a single equation as its induction formula.

For (34), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (35), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (35) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{38}$$

Indeed, both sides are $S(y + r)$.

For (36), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (37). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (36) and the induction hypothesis.

F.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{39}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (34) gives $0+x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (30).

For the step, assume (39) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (39) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (38) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (39). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{40}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (39). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (37) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{41}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{42}$$
$$e(a,b) = 0 \ \vee \ e(a,b) = 1. \tag{43}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (28), which is equivalent to $d(Sa,b) = 1$ by (31), and then to $a = b$ by (40). The range assertion is (27) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (42) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, \big(a = b \vee (a = b \to \bot)\big) \tag{44}$$

is a theorem, not an extra hypothesis.

F.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{45}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{46}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{47}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{48}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{49}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (42)–(43). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (49) for every quantifier-free formula.

F.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (49), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

G Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [6]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

G.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a+b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a+b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

G.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

G.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section C give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic.* Commit `62396b06020f794fa6dc01b4025a6a5db9d4d28a`, 12 November 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/62396b06020f794fa6dc01b4025a6a5db9d4d28a/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] Harvey Friedman. Provable equality in primitive recursive arithmetic with and without induction. *Pacific Journal of Mathematics* 57(2) (1975), 379–392. doi:10.2140/pjm.1975.57.379.

[3] ProveIt source series. Generated-assumption and constructed-computation presentations, inherited in `Research/certified_zero/reference/` at the pinned commit [1].

[4] ProveIt source series. Scalar predecessor bootstrap, reproduced as `scalar_predecessor.tex` in the pinned reference directory [1].

[5] ProveIt source series. Equality reflection in the forward-left presentation, reproduced as `equality_reflection.tex` in the pinned reference directory [1].

[6] ProveIt source series. Primitive-recursion compiler and pairing construction, reproduced as `primitive_recursion.tex` in the pinned reference directory [1].

[7] ProveIt source series. `Research/constructed_computation/`, at the pinned commit [1].

[8] ProveIt source series. `Research/transient_seed/`, at the pinned commit [1].